

AutoTestHub 踩坑记录与解决方案

面试复习专用 · 11 个真实踩坑案例 + 经验总结

1. 核心踩坑记录（按模块）

1.1 知识库 — pypdf 缺失导致异步向量化静默失败

现象：上传 PDF 文档后，知识库搜索不到内容，但页面显示「上传成功」。

根因：

- Celery 异步向量化任务因 pypdf 未安装而抛出 ImportError
- Celery 任务静默失败，没有回写错误状态到数据库
- 前端轮询到的任务状态始终是 SUCCESS，但实际上文档内容为空白

解决方案：

- 在 requirements.txt 中添加 pypdf>=3.0.0 依赖
 - 向量化任务增加 try-except 包裹，异常时回写 FAILED 状态
 - 前端展示任务失败原因，提示用户重新上传
-

1.2 知识库 — pymilvus 2.4 Hit 对象 API 变更

现象：升级到 pymilvus 2.4 后，搜索返回值类型不兼容，代码报 AttributeError。

根因：

- pymilvus 2.3 中 hit.get('entity') 返回 dict 类型
- pymilvus 2.4 中 hit.get('entity') 返回 NamedTuple 类型
- 代码用 dict['key'] 方式访问字段，在 2.4 中不再兼容

解决方案：

```
entity = hit.get('entity')
if isinstance(entity, dict):
    value = entity.get('field')
else:
    value = entity.field # NamedTuple 方式访问
```

- 用 isinstance 判断类型，兼容两种 API
-

1.3 知识库 — similarity_threshold=0.5 过滤过度

现象：知识库问答对概括性问题（如「这个项目是做什么的」）回答质量差，找不到相关内容。

根因：

- 向量相似度阈值设为 0.5，概括性问题与具体文档内容天然余弦相似度低 (0.3-0.4)
- 阈值一刀切，概括性问题和精确查询无法区分对待

解决方案：

- 将 `similarity_threshold` 从 0.5 降低到 0.0
 - 让 LLM 自行判断检索到的文档是否相关，而非在向量层面就过滤掉
-

1.4 API 路径不一致导致 404

现象：前端某些页面偶尔报 404，同样的功能在另一个页面却正常。

根因：

- 前端 `axios baseUrl` 和后端 URL 配置未统一管理
- 有的 API 用 `/api/knowledge-base/`，有的用 `/api/knowledge/knowledge-bases/`
- 前后端各自维护一套 URL，出现差异时难以发现

解决方案：

- 在统一的 API 常量文件中定义所有接口路径
 - 后端 Django URL 配置和使用同一套命名规范
 - 新增兼容路由（如 `/api/executions/` 同时映射到原有视图）
-

1.5 数据库字段设计 — `business_domain` 容量不足

现象：创建知识库时填入的场景描述（几十字）无法保存到 `business_domain` 字段。

根因：

- `business_domain` 定义为 `CharField(max_length=20)`，只能存 20 个字符
- 前端把完整场景描述（50+ 字符）塞进这个字段
- 本地 SQLite 不校验长度，到了服务器 PostgreSQL 才报错

解决方案：

- `business_domain` 保持 `varchar(20)`，只存简短枚举值（如 'AI评测'）
 - 长文本描述放入 `description` 字段（`TextField`，无长度限制）
-

1.6 开发环境与生产环境数据库不一致

现象：本地开发环境一切正常，部署到服务器后出现各种字段校验错误。

根因：

- SQLite 是「宽容模式」，不校验 `VARCHAR` 长度、不校验约束
- PostgreSQL 是「严格模式」，所有约束都会强制校验
- 开发阶段在 SQLite 上的操作没有触发约束检查

解决方案：

- 开发环境使用 `docker-compose.dev.yml` 启动 PostgreSQL 容器

- 所有开发操作都在 PostgreSQL 上进行，避免环境差异
-

1.7 Windows 端口冲突 (5432/5433)

现象：Windows 上启动 PostgreSQL Docker 时端口冲突，无法绑定 5432。

根因：

- Windows 系统预留了 5432-5433 端口给某些系统服务
- PostgreSQL 默认端口 5432 被占用，且不是已运行的 PostgreSQL 实例

解决方案：

- 将 PostgreSQL 端口映射改为 15432:5432
 - 更新 .env 中 DB_PORT=15432
-

1.8 安全意识 — .env 文件误提交 Git

现象：检查 GitHub 仓库时发现历史 commit 中包含 .env 文件，内有 API 密钥。

根因：

- 项目初期 .env 没有被 .gitignore 忽略，直接提交到了 Git 仓库
- 后期即使添加了 .gitignore，历史 commit 中的密钥仍然可被查阅
- 代码中还存在硬编码的默认密钥值 (seed_model_configs.py)
- HANDOFF.md 交接文档中记录了明文密钥

解决方案：

- 立即去云平台控制台重置所有泄露的 API Key (DashScope / DeepSeek)
 - .gitignore 添加 .env / .env.* 规则
 - 代码中所有 API Key 默认值改为空字符串：os.getenv('KEY', '')
 - 文档中所有密钥用 *** 占位符替代
 - Git 历史无法撤回，只能通过重置密钥来补救
-

1.9 数据库操作对象混淆 — 本地 Django ORM vs 服务器数据库

现象：用本地 Python 脚本创建的数据，服务器前端看不到。

根因：

- 本地 Django ORM 操作的是本地 PostgreSQL (或 SQLite) 数据库
- 服务器部署的是独立的 PostgreSQL 容器
- 两个数据库完全隔离，数据不会自动同步

解决方案：

- 数据操作统一通过 API 调用，而不是直接操作数据库
- 如需批量操作，使用 Django Management Command，在服务器容器中运行：

```
docker exec python manage.py
```

1.10 Celery 在 Windows 上的兼容性

现象：Windows 上启动 Celery worker 后任务不执行或报错。

根因：

- Celery 4.x+ 默认使用 prefork pool，Windows 不支持 fork 系统调用
- 某些依赖（如 sqlite3 扩展）在 Windows 上行为不同

解决方案：

- Windows 开发时使用 --pool=solo 启动：celery -A project worker -l info --pool=solo
- 生产环境（Linux Docker）使用默认 prefork pool，性能最优

1.11 国内网络导致模型下载失败

现象：首次运行时，sentence-transformers 下载模型 BAAI/bge-small-zh-v1.5 超时失败。

根因：

- Hugging Face 在国内访问不稳定，模型文件下载超时
- 没有配置镜像源或代理

解决方案：

- 优先使用 DashScope 在线 Embedding API（不需要下载模型）
- 如需本地 Embedding，设置 HF_ENDPOINT=https://hf-mirror.com 环境变量
- 编写下载重试逻辑和降级方案

2. 经验总结

2.1 环境一致性原则

开发环境、测试环境、生产环境必须使用相同的数据库和服务。SQLite 的宽容性会掩盖大量问题。推荐使用 `docker-compose.dev.yml` 在本地启动与生产一致的 PostgreSQL。

2.2 密钥管理铁律

规则	说明
绝不硬编码	os.getenv('KEY', '') 默认值为空，不给真值
.gitignore 先行	项目初始化第一步就加 .env
历史无法撤回	一旦泄露必须重置密钥，Git filter-repo 太麻烦
文档要脱敏	交接文档、README 中的密钥用 *** 替代

2.3 异步任务可靠性

规则	说明
显式异常捕获	所有 Celery 任务必须 try-except，不能指望框架兜底
状态回写	任务成功/失败都要写回数据库，前端依赖状态判断
超时配置	根据任务类型设置合理的 CELERY_TASK_TIME_LIMIT
幂等性	任务设计要考虑重试场景，不能重复写入数据

2.4 LLM 集成经验

经验	说明
阈值不宜过低	向量相似度阈值设为 0.0，让 LLM 自己判断相关性
兼容性检查	第三方 SDK 升级后必须验证 API 兼容性
降级方案	LLM 调用失败时有默认回答/本地模型兜底
模型管理化	模型配置存数据库而非代码，支持前端动态切换